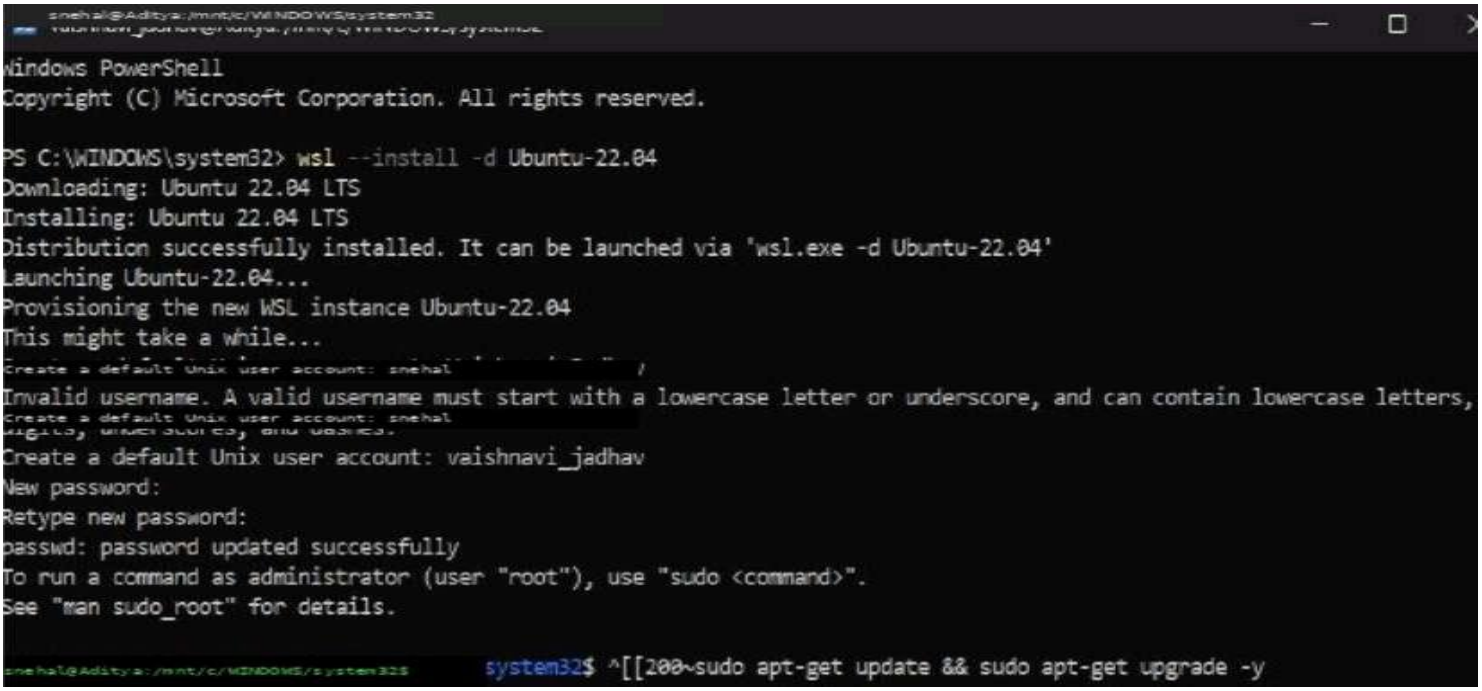
DevOps (MINI PROJECT 4)

NAME:-Mujmule Snehal
PRN:-2125UDSF1078

Step 0: Set up WSL2 on Windows

➢ Ubuntu 22.04 running under WSL2 on Windows. It is the Linux control node for Terraform and Ansible, because Ansible does not run on native Windows.



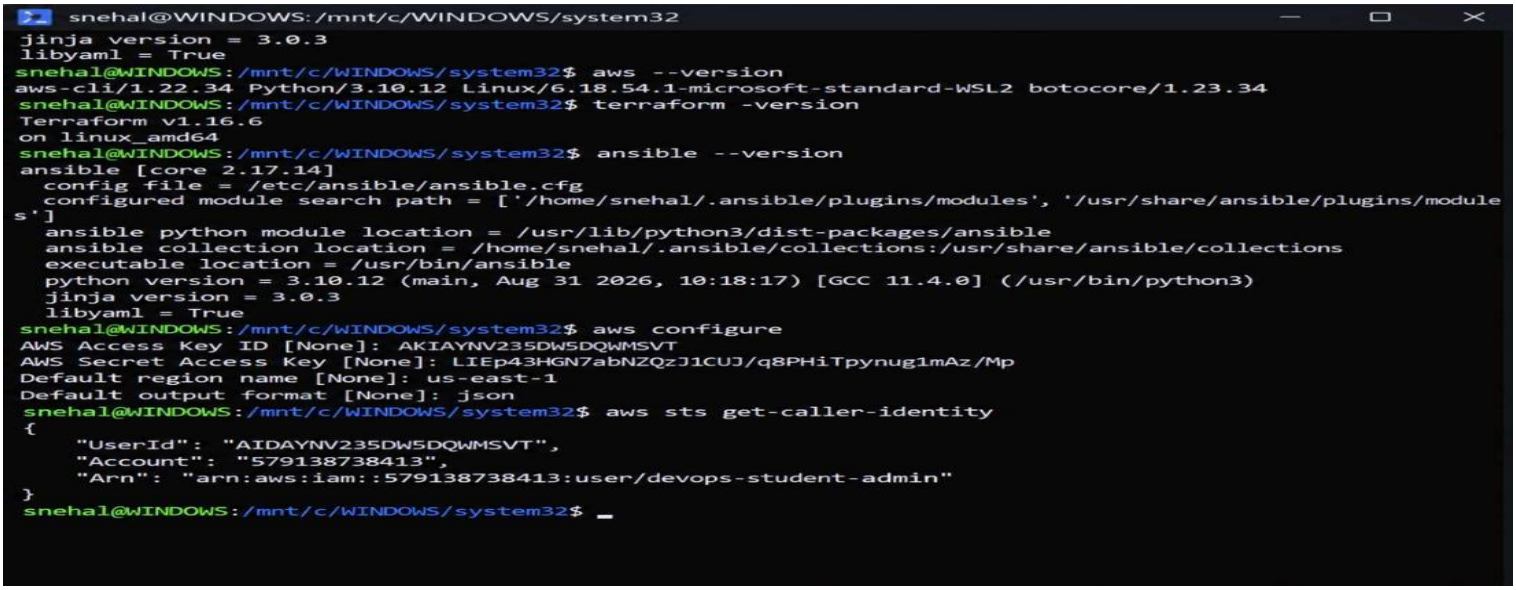
Step 1: Baseline tools, AWS CLI v2, Terraform, Ansible

➢ AWS CLI v2, Terraform and Ansible Core were installed on the control node. These three tools form the toolchain: Terraform builds the cloud infrastructure, Ansible configures the server, and the CLI connects both to AWS.



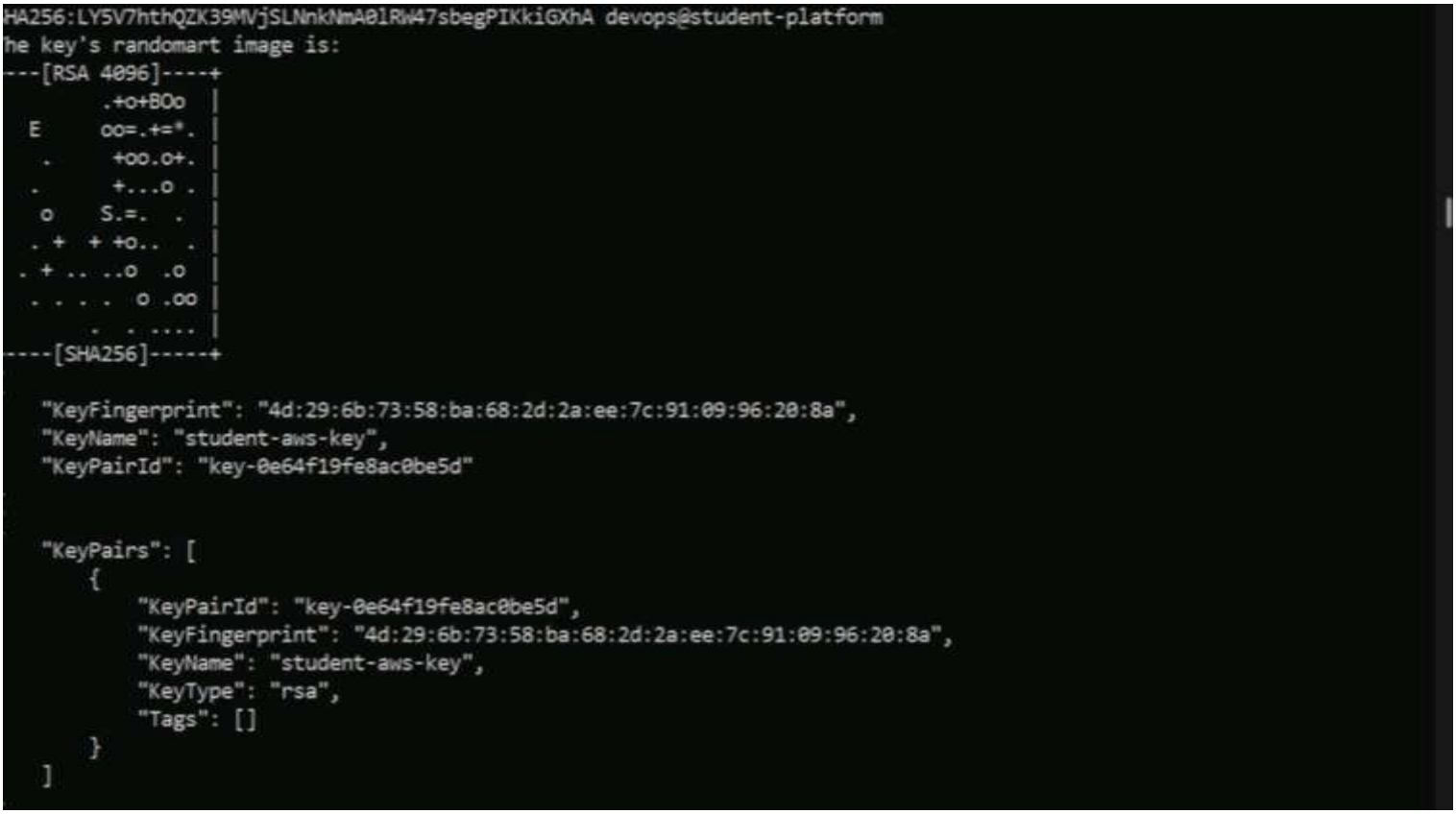
Step 2: Create an IAM user and access key

➢ AWS CLI authentication was configured with an IAM user. The calleridentity output confirms the credentials work, and Terraform will pick them up automatically from ~/.aws. No keys are written into the code



Step 3: Create and import the SSH key

➢ A 4096-bit RSA key pair was generated locally and only the public key was imported into AWS EC2. Ansible later uses the private key to log in to the server over SSH



Step 4: Copy the zip into WSL

➢ The project follows a standard Infrastructure-as-Code layout. The terraform folder defines the AWS resources, ansible holds the roles that configure the server, app is the Flask student-record application with its Dockerfile, scripts contains the inventory and validation helpers, and tests holds the automated unit tests

```
nt-infra
anzip -q student-record-infra-project.zip
:d student-infra
tudent-infra$ chmod +x scripts/*.sh
tudent-infra$ find . -type f | sort
/.gitignore
/ansible/ansible.cfg
/ansible/group_vars/all.yml
/ansible/group_vars/dev.yml
/ansible/group_vars/prod.yml
/ansible/inventory/.gitkeep
/ansible/roles/common/tasks/main.yml
/ansible/roles/docker/tasks/main.yml
/ansible/roles/nginx/handlers/main.yml
/ansible/roles/nginx/tasks/main.yml
/ansible/roles/nginx/templates/student-app.conf.j2
/ansible/roles/student_app/tasks/main.yml
/ansible/site.yml
/app/.dockerignore
/app/Dockerfile
/app/app.py
/app/requirements.txt
/pytest.ini
/scripts/gen_inventory.sh
/scripts/validate.sh
/terraform/main.tf
/terraform/outputs.tf
/terraform/variables.tf
/terraform/versions.tf
/tests/test_app.py
tudent-infra$
```

Step 5 : Run the unit tests locally

➢ The application's automated tests pass before deployment: health check, seeded students, duplicate and validation handling, and marks calculation. Testing before provisioning prevents deploying broken code.

```
  Downloading iniconfig-2.3.1-py3-none-any.whl (7.6 kB)
Collecting pluggy<2,>=1.5
  Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Collecting typing-extensions>=4.6.0
  Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45.6/45.6 KB 214.2 kB/s eta 0:00:00
Collecting MarkupSafe>=2.0
  Downloading markupsafe-3.0.4-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Installing collected packages: typing-extensions, tomli, pygments, pluggy, packaging, MarkupSafe, itsdangerous, iniconfi
g, click, blinker, Werkzeug, Jinja2, gunicorn, exceptiongroup, pytest, Flask
Successfully installed Flask-3.0.3 Jinja2-3.1.6 MarkupSafe-3.0.4 Werkzeug-3.1.9 blinker-1.9.0 click-8.5.0 exceptiongroup
-1.3.1 gunicorn-22.0.0 iniconfig-2.3.1 itsdangerous-2.2.0 packaging-26.3 pluggy-1.6.0 pygments-2.21.0 pytest-9.1.1 tomli
-2.5.0 typing-extensions-4.16.0
============================================== test session starts ==============================================
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0 -- /home/vaishnavi_jadhav/student-infra/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/vaishnavi_jadhav/student-infra
configfile: pytest.ini
testpaths: tests
collected 6 items

tests/test_app.py::test_health PASSED                                                                    [ 16%]
tests/test_app.py::test_seed_students PASSED                                                             [ 33%]
tests/test_app.py::test_add_student_and_duplicate PASSED                                                 [ 50%]
tests/test_app.py::test_add_student_validation PASSED                                                    [ 66%]
tests/test_app.py::test_marks_flow_and_percentage PASSED                                                 [ 83%]
tests/test_app.py::test_marks_rejects_bad_input PASSED                                                   [100%]

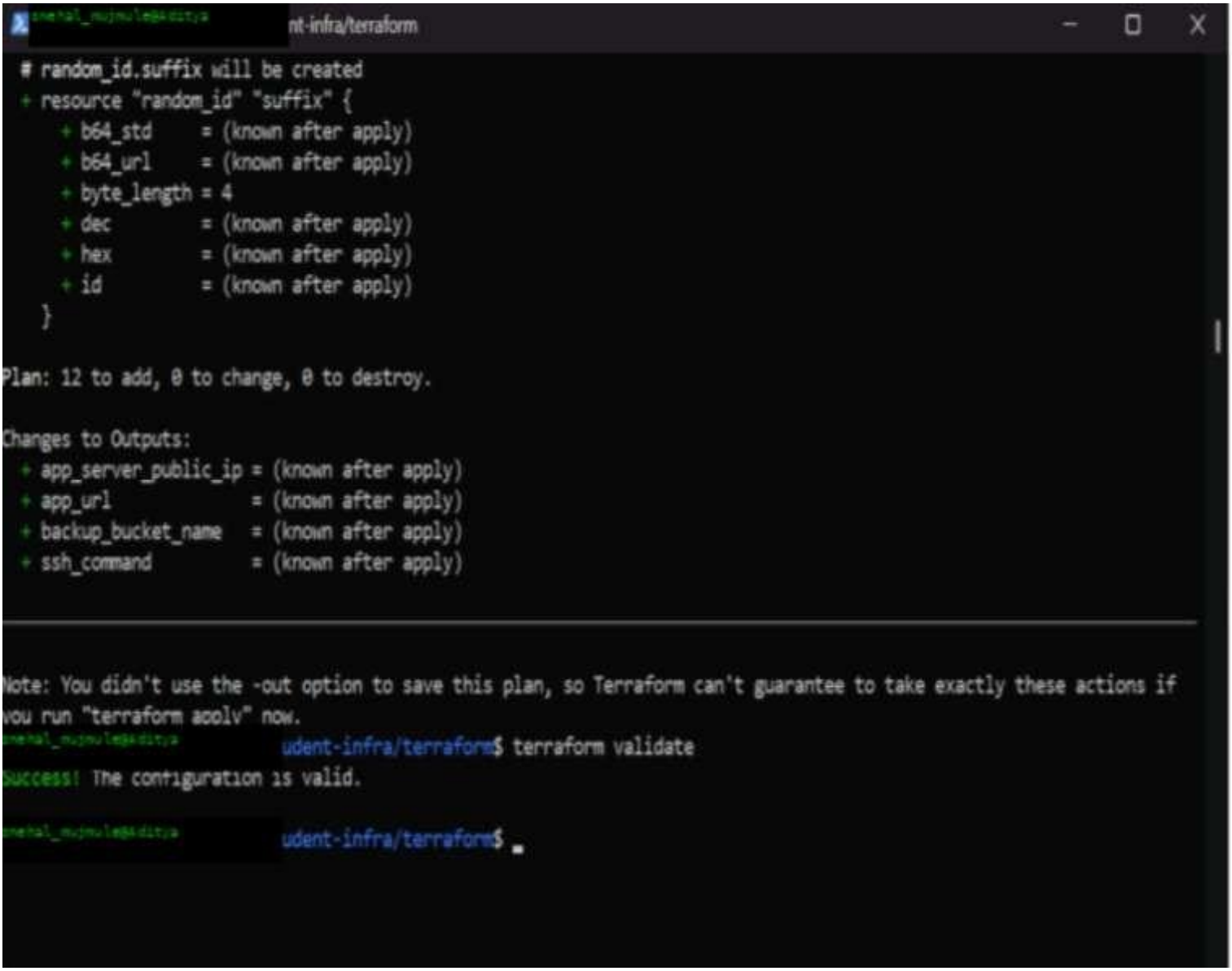
=============================================== 6 passed in 0.64s ===============================================
```

Step 6: Set your IP and create a workspace

➢ Restricting SSH to your own IP is safer than the default 0.0.0.0/0: cd ~/student-infra/terraform echo "admin_cidr = \"$(curl -s https://checkip.amazonaws.com)/32\"" > terraform.tfvars cat terraform.tfvars terraform workspace new dev terraform workspace show

Step 7: Init, validate, plan

➢ Terraform downloaded the AWS and random providers, and validation confirmed the HCL syntax is correct without calling AWS. The execution plan lists 12 resources to be created: VPC, internet gateway, public subnet, route table and association, security group, EC2 instance, S3 bucket with versioning, encryption and public-access block, and a random suffix. Reviewing the plan first shows exactly what will change



Step 8: Apply

➢ The AWS infrastructure was created from code. The outputs give the server's public IP, the application URL and the name of the S3 backup bucket

```
      + encrypted             = true
      + iops                  = (known after apply)
      + kms_key_id            = (known after apply)
      + tags_all              = (known after apply)
      + throughput            = (known after apply)
      + volume_id             = (known after apply)
      + volume_size           = 20
      + volume_type           = "gp3"
    }
  }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + app_server_public_ip = (known after apply)
  + app_url              = (known after apply)
  + ssh_command          = (known after apply)
aws_instance.app_server: Creating...
aws_instance.app_server: Still creating... [00m10s elapsed]
aws_instance.app_server: Creation complete after 19s [id=i-08e4dc884cdac318d]

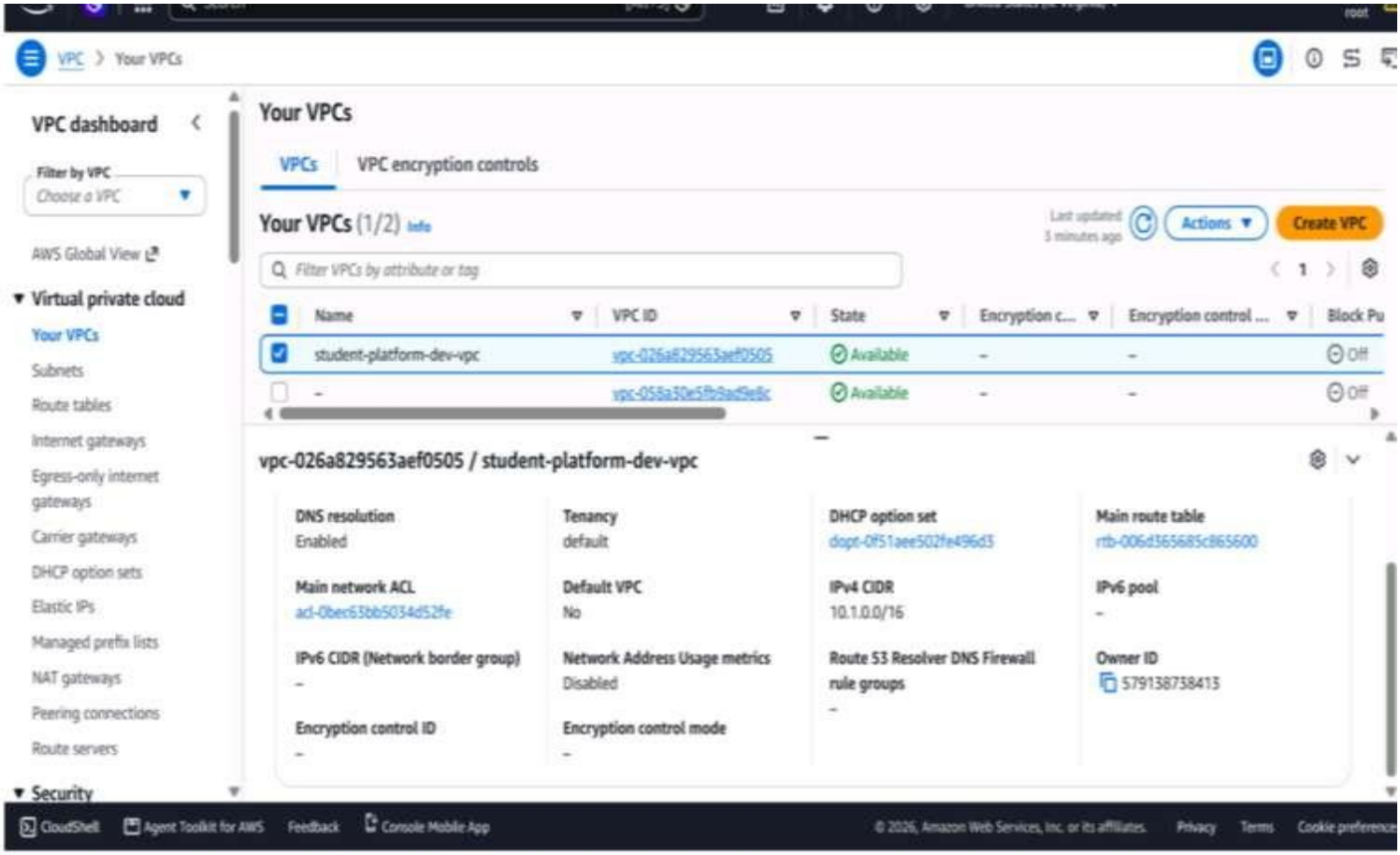
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

app_server_public_ip = "44.200.225.223"
app_url = "http://44.200.225.223"
backup_bucket_name = "student-platform-dev-backups-31bf6e4a"
ssh_command = "ssh -i ~/.ssh/student-aws-key ubuntu@44.200.225.223"
```

Step 9: Check in the AWS Console

➢ The EC2 instance created by Terraform is running inside the project's VPC, with the Environment tag set to the dev workspace. The dedicated VPC (10.1.0.0/16) and the private, encrypted, versioned S3 backup bucket exist as declared in the Terraform code



Step 10: Generate the inventory (the Terraform to Ansible handshake)

➢ A small script reads Terraform's output and writes the Ansible inventory automatically. This handshake passes the dynamically assigned public IP from the provisioning tool to the configuration tool, with no manual copying

```
./scripts/gen_inventory.sh
Inventory written for workspace 'dev' -> 44.200.225.223
[student_servers:children]
dev

[dev]
student-host ansible_host=44.200.225.223 ansible_user=ubuntu ansible_ssh_private_key_file=/home/vaishnavi_jadhav/.ssh/st
udent-aws-key
```

Step 11: Test the connection

➢ Ansible reached the new EC2 instance over SSH using the imported key. No agent is installed on the server, because Ansible only needs SSH and Python

```
ansible all -m ping
[WARNING]: Platform linux on host student-host is using the discovered Python interpreter at /usr/bin/python3.10, but
future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.html for more information.
student-host | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.10"
    },
    "changed": false,
    "ping": "pong"
}
```

Step 12: Run the playbook

➢ The playbook configured the server through four modular roles: base system and firewall, Docker, Nginx reverse proxy, and the containerized student-record application. A recap with failed=0 means every task succeeded.

```
ansible all -m ping
[WARNING]: Platform linux on host student-host is using the discovered Python interpreter at /usr/bin/python3.10, but
future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.html for more information.
student-host | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.10"
    },
    "changed": false,
    "ping": "pong"
}
```

Step 13: Run the four-gate validation

➢ The four-gate validation passed: network and OS (SSH reachable), daemons and ports (Docker and Nginx active, port 80 open), application container (running, database file present), and API smoke tests (health returns UP, students count is at least 2, POST marks returns 201). The saved log is included as evidence



Step 14: Open the app in your browser

➢ The containerized Student Record application is live on AWS and reachable on port 80 through the Nginx reverse proxy. The REST API returns student records from the persistent SQLite database, confirming the application, container, and storage work end to end

```
ok: [student-host]

TASK [student_app : Wait for the application port] ********************************************************************
ok: [student-host]

TASK [student_app : Verify the health endpoint through Nginx] ********************************************************
ok: [student-host]

PLAY RECAP ***********************************************************************************************************
student-host               : ok=19   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

vaishnavi_jadhav@Aditya:~/student-infra/ansible$ cd ~/student-infra
./scripts/validate.sh $(cd terraform && terraform output -raw app_server_public_ip) | tee validation-log.txt
=== Gate 1: Network & OS ===
  [PASS] SSH port 22 reachable
  [PASS] Ubuntu host responds (uptime)
=== Gate 2: Daemons & Ports ===
  [PASS] Docker daemon active
  [PASS] Nginx service active
  [PASS] Port 80 reachable
=== Gate 3: App Container ===
  [PASS] Container student-app is Up
  [PASS] SQLite database file exists
=== Gate 4: API Smoke Tests (through Nginx on port 80) ===
  [PASS] GET /api/health returns UP
  [PASS] GET /api/students count >= 2
  [PASS] POST /api/marks returns 201

RESULT: ALL GATES PASSED
```

```
snehal@WINDOWS:~$ /scripts/validate.sh
== Gate 1: Network & OS ===
[PASS] SSH port 22 reachable
[PASS] Ubuntu host responds (uptime)
== Gate 2: Daemons & Ports ===
[PASS] Docker daemon active
[PASS] Nginx service active
[PASS] Port 80 reachable
== Gate 3: App Container ===
[PASS] Container student-app is Up
[PASS] SQLite database file exists
== Gate 4: API Smoke Tests (through Nginx on port 80) ===
[PASS] GET /api/health returns UP
[PASS] GET /api/students count >= 2
[PASS] POST /api/marks returns 201

RESULT: ALL GATES PASSED

snehal@WINDOWS:~/student-infra/ansible$ cat /scripts/validate.sh
#!/bin/bash
# Script to validate the deployed application - 4 gates

APP_URL="http://<your-public-ip>/api/health"

LOG_FILE="validation-log.txt"

log() {
   echo "[$(date '+%F %T')] $1" | tee -a "$LOG_FILE"
}

snehal@WINDOWS:~/student-infra/ansible$
```

{"service":"Student Record Platform","status":"UP","version":"1.0.0"}

{"count":2,"students":[{"course":"Computer Science","id":1,"name":"Aarav Sharma","roll_no":"CS10
{"course":"Computer Science","id":2,"name":"Diya Patel","roll_no":"CS102"}]}